

STOCK MARKET TRADING WEB APPLICATION USING MERN STACK (ZERODHA CLONE)

A PROJECT REPORT

Submitted By

Vipin Pal – 2203451650026
Aman Maurya – 2203451650005
Udit Narayan Maurya – 2203451650025
Paritosh Kumar Sahu – 2203451650012

Project-II (BCS 851)

UNDER THE SUPERVISION OF

Ms. Sakshi Raj Singh

Assistant Professor

(Computer Science & Engineering)

HEAD OF DEPARTMENT

Dr. Arvind Kumar Singh

Associate Professor

(Computer Science & Engineering)

SETH VISHAMBHAR NATH INSTITUTE OF ENGINEERING & TECHNOLOGY, SAFEDABAD, BARABANKI

in partial fulfillment of the requirements for the award of the Degree
of
Bachelor of Technology
In



Computer Science & Design

(Department Of Computer Science & Engineering)

Affiliated To



DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY LUCKNOW Batch

2022-26

CERTIFICATE

This is to certify that the project entitled “**Stock Market Trading Web Application Using MERN Stack (Zerodha Clone)**” submitted by **Vipin Pal, Aman Maurya, Udit Narayan Maurya, Paritosh Kumar Sahu** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Design** from **Dr. A.P.J. Abdul Kalam Technical University** is a record of the candidate’s original work carried out under my supervision and guidance.

To the best of my knowledge and belief, the work embodied in this project report has not been submitted previously for the award of any other degree or diploma to this or any other university or institution. The project report reflects the results of the student's independent work and studies carried out during the course of the project.

Project Coordinator

Mr. Jay Kumar

Assistant Professor

Head of Department

Dr. Arvind Kumar Singh

DECLARATION

We hereby declare that the project entitled “Stock Market Trading Web Application Using MERN Stack (Zerodha Clone) submitted by us in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (Computer Science and Design) from Seth Vishambhar Nath Institute of Engineering & Technology, Barabanki is a record of our own work carried out under the supervision and guidance of Ms. Sakshi Raj Singh (Assistant Professor).

To the best of our knowledge, this project has not been submitted to any other University or Institute for the award of any degree.

Submitted By:

Vipin Pal – 2203451650026

Aman Maurya – 2203451650005

Udit Narayan Maurya – 2203451650025

Paritosh Kumar Sahu – 2203451650012

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to our respected guide Ms. **Sakshi Raj Singh (Assistant Professor)** Department of **Computer Science and Engineering**, **Seth Vishambhar Nath Institute of Engineering & Technology**, for their valuable guidance, continuous encouragement, and support throughout the completion of this project.

I am also thankful to the Head of the Department, Computer Science and Engineering, and all the faculty members of the department for their cooperation and valuable suggestions during the development of this project.

I would like to express my deep sense of gratitude to the management of Seth Vishambhar Nath Institute of Engineering & Technology for providing the necessary facilities and a supportive environment to complete this work successfully.

Finally, I would like to thank my parents, friends, and classmates for their constant motivation, encouragement, and support during the entire period of this project work.

Student Name

Guide Name

Vipin Pal – 2203451650026

Ms. Sakshi Raj Singh

Aman Maurya – 2203451650005

Assistant Professor

Udit Narayan Maurya – 2203451650025

Paritosh Kumar Sahu – 2203451650012

ABSTRACT

This project focuses on the design and development of a full-stack stock trading web application inspired by Zerodha, one of India's leading online trading platforms. The main objective of this project is to create a user-friendly and efficient system that allows users to simulate stock trading activities, manage their portfolios, and analyze market trends in a secure and interactive environment. The application is developed using the MERN stack, which includes MongoDB for database management, Express.js and Node.js for backend development, and React.js for building a responsive and dynamic user interface.

The platform provides essential features such as user authentication and authorization, real-time/simulated stock price updates, portfolio management, and an interactive dashboard. Users can register and log in securely, view available stocks, and perform buy and sell operations. The system maintains records of user transactions and dynamically updates their portfolio based on trading activities. Additionally, graphical representations, such as charts and analytics, are integrated to help users understand market trends and make informed decisions.

On the backend, RESTful APIs are implemented to handle data processing, user requests, and client-server communication. Proper validation and error handling mechanisms are incorporated to ensure data integrity and system reliability. The frontend is designed with a focus on usability and responsiveness, ensuring that the application works smoothly across different devices and screen sizes.

This project demonstrates the practical application of full-stack web development concepts and highlights the importance of scalable architecture in building modern financial applications. It also provides hands-on experience with real-world technologies and tools used in the industry. Overall, the project serves as a learning platform for understanding the workflow of stock trading systems and showcases the ability to develop a complete web-based solution using modern technologies.

CONTENTS

CERTIFICATE

DECLARATION

ACKNOWLEDGMENT

ABSTRACT

1	Introduction	1
1.1	Background of the study.....	2
1.2	Problem Statement.....	3
1.3	Objectives of the Project.....	4
1.4	Scope of the Project.....	5
2	Literature Review.....	8
2.1	Previous Research.....	10
2.2	Journals and Similar Systems.....	10
3	System Analysis and Design.....	12
3.1	System Analysis.....	13
3.2	Proposed System.....	13
3.3	Feasibility Study.....	14
3.4	Requirement Analysis.....	16
3.5	System Design.....	18
3.6	Architecture Design.....	19
3.7	Module Design.....	23
3.8	Database Design.....	26
3.9	Process Flow of System.....	27
3.10	Testing Strategy.....	29
3.11	Advantages of Designed System.....	29
3.12	Future Enhancements.....	29
4	Implementation.....	31

4.1 Implementation.....	31
4.2 API Integration.....	33
4.3 Testing and Debugging.....	34
4.4 Deployment.....	35
5 Source Code.....	36
6 Results and Discussion.....	62
6.1 Results	62
6.2 Discussions.....	66
7 Future Scope.....	68
8 Conclusion.....	69
9 References.....	70
9.1 Books & Study Materials.....	70
9.2 Web Technologies Documentation.....	70
9.3 Artificial Intelligence & Machine Learning Resources.....	70

LIST OF FIGURES

1. Figure: 3.3.1 Data Flow Diagram.....	29
2. Figure: 3.3.2 Level 0 & Level 1 DFD.....	30
3. Figure: 6.1.11 Home Page.....	69
4. Figure: 6.1.11 Registration.....	70
5. Figure: 6.1.3. Login.....	70
6. Figure: 6.1.4. Login Dashboard.....	71
7. Figure: 6.1.5. Stock.....	72
8. Figure: 7.1.3. Stock Chart.....	73

LIST OF ABBREVIATIONS

1. MERN.....MongoDB, Express.js, React.js, Node.js
2. API.....Application Programming Interface
3. REST.....Representational State Transfer
4. UI.....User Interface
5. UX.....User Experience
6. DFD.....Data Flow Diagram
7. JWT.....JSON Web Token
8. NLP.....Natural Language Processing
9. AI.....Artificial Intelligence
10. ML.....Machine Learning
11. HTTP.....Hypertext Transfer Protocol
12. HTTPS.....Hypertext Transfer Protocol Secure
13. CRUD.....Create, Read, Update, Delete
14. UAT.....User Acceptance Testing
15. DB.....Database
16. NoSQL.....Not Only Structured Query Language
17. JSON.....JavaScript Object Notation
18. 2FA.....Two-Factor Authentication

LIST OF NOTATIONS

1. → Data flow direction
2. DB Database storage
3. Txn Transaction
4. Auth Authentication
5. API Communication between frontend and backend
6. UI User Interface layer

CHAPTER 1: INTRODUCTION

The rapid advancement of technology has significantly transformed various sectors, including the financial industry. One of the most notable developments is the emergence of online stock trading platforms, which have made investing in the stock market more accessible, efficient, and user-friendly. In the past, stock trading required the involvement of intermediaries such as brokers and was often limited by time constraints and geographical barriers. However, with the rise of web-based applications, users can now trade stocks anytime and from anywhere using digital platforms.

Stock trading applications have gained immense popularity due to their ability to provide real-time market data, seamless transaction processing, and advanced analytical tools. These platforms enable users to monitor stock prices, manage their investment portfolios, and make informed financial decisions. The growing number of retail investors and the increasing demand for digital solutions have further accelerated the development of such systems.

This project focuses on the design and development of a stock trading web application inspired by modern platforms like Zerodha. The application aims to replicate key features such as user authentication, stock listing, buying and selling of shares, portfolio management, and data visualization through charts and dashboards. By implementing these functionalities, the project provides a simplified yet effective model of a real-world trading system.

The application is developed using the MERN stack, which consists of MongoDB, Express.js, React.js, and Node.js. This technology stack is widely used for building scalable and high-performance web applications. MongoDB is used for efficient data storage, Express.js and Node.js handle server-side operations and API development, while React.js is used to create an interactive and responsive user interface.

The primary goal of this project is to gain practical knowledge of full-stack web development and understand the working principles of online trading systems. It also aims to demonstrate how modern technologies can be used to build secure, scalable, and user-centric applications. Overall, this project serves as a comprehensive learning experience and provides insights into the development of real-world financial platforms.

Security and reliability are critical aspects of any financial application. In this project, appropriate measures are considered to ensure secure user authentication and data protection. Techniques such

as password encryption, token-based authentication, and secure API handling are implemented to safeguard sensitive user information. Additionally, the system is designed to handle multiple user requests efficiently, ensuring smooth performance even under high load conditions. This highlights the importance of building robust and secure backend systems in real-world applications.

1.1 Background of the Study

In recent years, the financial market has witnessed significant growth due to the rapid advancement of technology and increased internet accessibility. Online stock trading platforms have become an essential part of the modern financial ecosystem, allowing individuals to participate in the stock market with ease and convenience. Earlier, stock trading was limited to brokers and required physical presence or telephonic communication, which made the process time-consuming and less accessible. However, with the emergence of digital trading platforms, the entire process has become faster, more transparent, and user-friendly.

Modern trading applications provide a wide range of features such as real-time market data, portfolio tracking, advanced charting tools, and secure transaction systems. These platforms not only simplify the trading process but also empower users to make informed investment decisions. As a result, there has been a significant increase in the number of retail investors participating in the stock market.

The development of full-stack web technologies has further contributed to the growth of such platforms. Technologies like MongoDB, Express.js, React.js, and Node.js (MERN stack) enable developers to build scalable, efficient, and responsive applications. These technologies support dynamic user interfaces, real-time data handling, and secure backend operations, making them ideal for developing financial applications.

This project is inspired by the functionality and design of modern trading platforms and aims to replicate their core features in a simplified manner. The purpose of this study is to understand how a stock trading system works and to implement its fundamental components using modern web development technologies. By developing this application, the study explores key concepts such as user authentication, data management, real-time updates, and interactive dashboards.

Overall, this study highlights the importance of technology in transforming traditional trading systems into modern digital platforms. It also emphasizes the role of full-stack development in creating efficient and scalable web applications that meet the needs of today's users.

Another important aspect of the application is scalability and maintainability. The use of the MERN stack allows the system to be easily extended and modified as requirements evolve. Modular coding practices and well-structured APIs ensure that new features can be integrated without affecting existing functionalities. This makes the application adaptable to future technological advancements and increasing user demands, which is essential for any modern web-based platform.

In addition, the project highlights the importance of data visualization in financial decision-making. By incorporating charts and graphical representations, users can better understand market trends and analyze their investment performance.

1.2 Problem Statement

In today's digital era, although online stock trading platforms are widely available, many of them are complex and difficult for beginners to understand and use effectively. New users often face challenges in navigating trading interfaces, interpreting market data, and making informed decisions. Most existing platforms are designed for experienced traders and include advanced features such as technical charts and financial terminologies, which can be overwhelming for beginners and limit their practical exposure.

Another major issue is that real trading platforms involve financial risk, discouraging users from experimenting and learning through practice. There is a lack of safe and simulated environments where users can perform trading activities without the fear of losing money. As a result, many learners rely only on theoretical knowledge, which is insufficient to understand the real-world functioning of stock trading systems.

Moreover, existing systems often fail to provide a simplified and educational approach. Core functionalities such as portfolio management, transaction tracking, and performance analysis are usually presented in a complex manner, making them difficult for users to interpret. This highlights the need for a platform that balances functionality with simplicity, enabling users to learn and practice trading concepts in an interactive and user-friendly environment.

From a technical perspective, developing such a system presents several challenges, including handling dynamic data, ensuring secure user authentication, managing real-time updates, and maintaining system performance and reliability. Without proper implementation, issues such as data inconsistency, slow response time, and security vulnerabilities may arise.

Additionally, there is an increasing demand for scalable, responsive, and high-performance web applications that deliver a seamless experience across multiple devices. Many traditional systems fail to meet these expectations due to outdated technologies and poor system architecture, making the adoption of modern full-stack technologies essential.

Another important limitation is the lack of personalization and guided learning support in existing platforms. Beginners often struggle due to the absence of step-by-step guidance, tutorials, and intelligent recommendations that help them understand trading strategies and market behavior. This reduces user engagement and makes the learning process more difficult.

To address these challenges, this project proposes the development of a simplified stock trading web application inspired by platforms like Zerodha. The system focuses on providing an intuitive user interface, essential features such as user authentication, stock viewing, buying and selling of shares, and portfolio management, all within a simulated environment.

The application is developed using the MERN stack (MongoDB, Express.js, React.js, Node.js) to ensure scalability, efficiency, and security. It supports smooth frontend-backend communication, reliable data management, and a responsive user experience.

Overall, the problem addressed in this project is the lack of a simple, educational, and user-friendly stock trading system that enables users to learn and practice trading concepts effectively. This project aims to bridge the gap between theoretical knowledge and practical implementation, while demonstrating the use of modern technologies in building robust and scalable applications.

1.3 Objectives of the project

The primary objective of this project is to design and develop a full-stack stock trading web application inspired by modern trading platforms like Zerodha. The system aims to provide a simplified, interactive, and user-friendly environment where users can understand and perform stock trading activities efficiently.

One of the key objectives is to implement secure user authentication and authorization, ensuring that user data protection is maintained and only authorized users can access their accounts and perform trading operations.

Another objective is to develop a system that presents stock market data in a clear and organized manner, allowing users to view stock listings, analyze price information, and understand market trends to make informed decisions.

The project also aims to implement buying and selling functionalities within a simulated environment, enabling users to practice trading without financial risk. This is especially beneficial for beginners and students as it provides practical learning experience.

An important objective is to provide portfolio management, where users can track their transactions, monitor holdings, and evaluate their investment performance. This helps improve decision-making skills and understanding of investment patterns.

The project focuses on developing an interactive and responsive user interface that ensures compatibility across multiple devices such as desktops, tablets, and smartphones, thereby enhancing the overall user experience.

Another objective is to integrate data visualization tools such as charts and dashboards, which help users understand stock trends, price fluctuations, and overall market behavior effectively.

From a technical perspective, the project aims to utilize the MERN stack (MongoDB, Express.js, React.js, Node.js) to build a scalable, efficient, and robust application. This includes RESTful API development, database management, and ensuring smooth frontend-backend communication.

The project also emphasizes data validation, error handling, and system reliability to maintain data integrity and prevent system failures. Additionally, security practices are implemented to protect sensitive user information.

Finally, the project aims to follow modular coding practices and reusable code structure, which enhance maintainability, simplify debugging, and support future scalability. This reflects the importance of clean coding standards and software design principles in building professional web applications.

1.4 Scope of the project

The scope of this project focuses on the design and development of a web-based stock trading application inspired by modern trading platforms. The application aims to provide a simplified and interactive environment where users can understand and perform basic stock trading operations. It

is primarily developed for educational and demonstration purposes, allowing users to explore the functionalities of a trading system without involving real financial transactions.

The project covers the development of essential features required in a stock trading platform. These include user registration, authentication, stock listing, buying and selling of stocks, and portfolio management. Users can create an account, log in securely, and access their personalized dashboard. The system maintains user-specific data such as transaction history and portfolio details, ensuring a customized experience for each user.

One of the important aspects within the scope of this project is the implementation of a simulated trading environment. Instead of using real money, the application allows users to perform trading operations in a virtual setting. This helps beginners and students to learn trading concepts, understand market behavior, and practice decision-making without financial risk. The simulation provides a practical understanding of how stock trading platforms function in real-world scenarios.

The project also includes the integration of dynamic data handling and user interaction features. Stock data is displayed in a structured format, allowing users to view price changes and other relevant information. Additionally, graphical representations such as charts and dashboards are included to enhance data visualization. These features help users analyze trends and gain insights into market patterns in an easy and understandable manner.

From a technical perspective, the scope involves the use of the MERN stack, which consists of MongoDB, Express.js, React.js, and Node.js. The frontend is designed to be responsive and interactive, ensuring a smooth user experience across different devices. The backend is responsible for handling business logic, API development, and database communication. MongoDB is used to store and manage user data, transaction records, and other application-related information efficiently.

The project also focuses on implementing secure and reliable system operations. User authentication mechanisms are designed to protect sensitive data and ensure that only authorized users can access the system. Validation and error-handling techniques are incorporated to maintain system stability and prevent unexpected failures. These measures contribute to the overall reliability and usability of the application.

In terms of scalability, the project is designed to allow future enhancements and feature additions. The system can be extended to include real-time stock market integration, payment gateway

support, advanced analytics, and mobile application support. This makes the project flexible and adaptable to future requirements.

However, the scope of this project is limited to a simplified version of a stock trading platform. It does not involve real-time financial transactions, live trading, or banking system integration. The application is intended for learning purposes only and does not aim to replace professional trading platforms. Advanced features such as high-frequency trading, algorithmic trading, and regulatory compliance are out of scope.

Additionally, the project is developed as a web-based application and does not include native mobile app development. The focus remains on full-stack web development concepts rather than building a production-level financial system. The design prioritizes simplicity, clarity, and ease of use.

Overall, the scope of this project lies in developing a functional, user-friendly, and educational stock trading web application that demonstrates the core concepts of trading systems. It provides a foundation for learning and offers opportunities for future improvements and expansion.

In addition, the scope includes a well-structured user interface design following modern UI/UX principles. The application emphasizes clarity, consistency, and ease of use, ensuring smooth user interaction. Features like intuitive controls, proper layout structuring, and feedback messages enhance usability.

Another enhancement includes role-based system behavior in a simplified form. The system can be extended to support administrative controls, such as managing stock listings and monitoring user activities. This introduces the concept of access control, which is essential in real-world applications.

CHAPTER 2: LITERATURE REVIEW

The development of online stock trading platforms has been widely studied due to the rapid growth of digital financial systems and increasing participation of retail investors. Various researchers and developers have explored the design, functionality, and performance of web-based trading applications, focusing on improving accessibility, usability, and system efficiency. This section reviews the existing concepts, technologies, and approaches related to stock trading systems and full-stack web application development.

In earlier studies, stock trading systems were primarily dependent on traditional brokerage methods, where transactions were carried out through intermediaries. These systems were often time-consuming and lacked transparency. With the introduction of online trading platforms, researchers highlighted the importance of digital transformation in financial markets. Online platforms enabled users to trade directly, access real-time market information, and manage their investments efficiently. This shift has significantly influenced interaction with financial markets.

Several studies emphasize the importance of user interface design (UI design) in trading applications. A well-designed interface enhances user experience (UX) and helps users understand complex financial data. Research suggests that simplified dashboards, intuitive navigation, and data visualization play a crucial role in improving user engagement. Modern platforms include charts, graphs, and analytical tools to support informed decision-making.

Another important aspect discussed in the literature is real-time data processing in stock trading systems. Many applications rely on real-time or near real-time updates to provide accurate stock prices and market trends. Researchers have explored methods such as API integration and data streaming techniques for handling dynamic data. Although real-time systems offer high accuracy, they require a robust backend infrastructure to manage continuous data flow and maintain system stability.

The adoption of full-stack development technologies has been widely discussed in recent studies. The MERN stack (MongoDB, Express.js, React.js, Node.js) has gained popularity for building scalable and high-performance web applications. Research indicates that this stack supports efficient data management, modular development, and responsive user interfaces. It enables developers to build complete applications using JavaScript, reducing complexity and improving development speed.

Security is another critical area highlighted in the literature. Stock trading platforms deal with sensitive user data and financial transactions, making them targets for cyber threats. Studies suggest implementing strong authentication mechanisms, data encryption, and secure API communication to ensure data protection. Even in a simulated environment, the importance of security practices remains essential in system design.

Several researchers have also focused on the educational aspects of trading platforms. Simulated trading systems or paper trading platforms help beginners learn trading without financial risk. These systems replicate real-world trading scenarios and allow users to trade using virtual money. Such platforms help bridge the gap between theoretical knowledge and practical experience.

Performance and scalability are key considerations in trading system development. As the number of users increases, systems must handle multiple concurrent requests without performance degradation. Techniques such as load balancing, efficient database design, and optimized API handling are used to improve system performance. The use of modern frameworks and cloud-based solutions further enhances the scalability of web applications.

Furthermore, existing literature highlights the importance of modular and maintainable system architecture. Breaking down applications into smaller components improves code readability, reusability, and ease of maintenance. This approach is especially useful in full-stack applications, where frontend and backend components must interact seamlessly.

Despite the availability of advanced trading platforms, studies indicate that many systems remain complex for beginners and lack a learning-oriented approach. This creates a need for simplified applications that focus on core functionalities and provide an easy entry point for new users. The current project addresses this gap by developing a user-friendly stock trading application with essential features and a simulated environment.

Overall, the literature review shows that online stock trading systems have evolved significantly with the advancement of modern technology. The integration of real-time data, interactive interfaces, and full-stack development frameworks has improved efficiency and accessibility. At the same time, there is a growing need for simplified and educational systems that help users understand trading concepts. This project builds upon existing research and aims to provide a practical solution using modern web technologies.

2.1 Previous Research

Previous research in the field of stock trading systems shows that technology plays a major role in improving trading efficiency and accuracy. Many studies focus on algorithmic trading, where computer programs automatically execute buy and sell orders based on predefined rules. This approach reduces human error and increases transaction speed. Researchers have found that automated systems can analyze large volumes of market data and make quick decisions.

Other research highlights the use of machine learning techniques in stock market prediction. Studies in Machine Learning show that models like neural networks and regression algorithms can predict stock price movements using historical data. These systems enhance decision-making by providing insights into future trends.

In addition, research on online trading platforms such as Zerodha explains how modern systems provide real-time data, portfolio management, and fast order execution. These platforms are designed to be user-friendly and efficient, allowing investors to trade easily from anywhere.

Some studies also focus on high-frequency trading (HFT), where trades are executed in milliseconds to take advantage of small price changes. While this improves market efficiency, it may also lead to increased market volatility.

Overall, previous research indicates that stock trading systems are becoming more advanced with the integration of automation, data analysis, and intelligent decision-making techniques.

2.2 Journals and Similar Systems

Various journals and research studies have discussed the development and impact of online stock trading systems. A study in the Journal of Management Research and Analysis explains that online trading platforms have significantly improved convenience, speed, and accessibility, making trading easier through internet-based systems. Another study in Computers & Mathematics with Applications highlights virtual stock trading systems, which simulate real market environments and enable learning without financial risk.

Recent studies also focus on user behavior and system design. Research in Electronic Commerce Research shows that user interaction, navigation patterns, and platform usability play a major role in the adoption of trading applications. A recent (2025) study emphasizes that user interface (UI), ease of use, and real-time data strongly influence user satisfaction and platform usage.

In terms of similar systems, platforms like Zerodha and other online brokerage applications provide features such as real-time trading, portfolio management, and fast order execution. Research also highlights the integration of decision support systems and automation, which continuously improve trading performance.

- Studies explain the use of algorithmic trading for fast and accurate transactions.
- Journals highlight the role of Machine Learning in predicting stock prices.
- Research focuses on user-friendly interface design and system design for better user experience (UX).
- Some studies discuss virtual trading systems used for learning without real money (risk-free environment).
- Journals mention the importance of real-time data updates in trading platforms.

CHAPTER 3: SYSTEM ANALYSIS & DESIGN

The development of online stock trading platforms has been widely studied due to the rapid growth of digital financial systems and increasing participation of retail investors. Researchers and developers have explored the design, functionality, and performance of web-based trading applications, focusing on improving accessibility, usability, and system efficiency. This section reviews key concepts, technologies, and approaches related to stock trading systems and full-stack web application development.

In earlier studies, trading systems relied on traditional brokerage methods, where transactions were handled by intermediaries. These systems were time-consuming and lacked transparency. With the emergence of online trading platforms, the importance of digital transformation in financial markets was highlighted. These platforms allow users to trade directly, access real-time market information, and manage investments efficiently, changing how users interact with financial markets.

Several studies emphasize user interface (UI) design in trading applications. A well-designed interface improves user experience (UX) and helps users understand complex financial data. Features like simplified dashboards, intuitive navigation, and data visualization increase user engagement. Modern platforms use charts, graphs, and analytical tools to support informed decision-making.

Another key aspect is real-time data processing in trading systems. Applications rely on real-time or near real-time updates for accurate stock prices and market trends. Researchers use API integration and data streaming techniques to handle dynamic data. While these systems provide high accuracy, they require a robust backend infrastructure to ensure continuous data flow and maintain system stability.

The adoption of full-stack development technologies has been widely discussed in recent studies. The MERN stack (MongoDB, Express.js, React.js, Node.js) is popular for building scalable and high-performance web applications. It supports efficient data management, modular development, and responsive user interfaces. Developers can build complete applications using JavaScript, which reduces complexity and improves development speed.

Security is another critical area highlighted in the literature. Stock trading platforms handle sensitive user data and financial transactions, making them targets for cyber threats. Studies

recommend strong authentication mechanisms, data encryption, and secure API communication to ensure data protection. Even in a simulated environment, the importance of security practices remains essential in system design.

3.1 System Analysis

Overall, the literature review shows that online stock trading systems have evolved significantly with the advancement of modern technology. The integration of real-time data, interactive interfaces, and full-stack development frameworks has improved the efficiency and accessibility of these platforms.

At the same time, there is a growing need for simplified and educational systems that help users understand trading concepts. This project builds upon these existing studies and aims to implement a practical solution using modern web technologies.

This traditional method has several drawbacks:

- Uses online trading platforms (mobile apps & web applications) for buying and selling stocks
- Provides real-time stock price updates and market data
- Includes features like portfolio management, charts, and order placement
- Depends on third-party APIs for stock data and payment processing
- Requires internet connectivity for all operations

Due to these limitations, there is a need for an intelligent and automated system.

3.2 Proposed System

The proposed system is a web-based stock trading application designed to provide a simplified and user-friendly platform for understanding and performing stock trading activities. The system is developed using the MERN stack (MongoDB, Express.js, React.js, Node.js) and focuses on delivering core functionalities such as user authentication, stock viewing, trading operations, and portfolio management within a simulated environment. It aims to overcome the complexity of existing platforms by offering an intuitive interface and a structured workflow, making it suitable

for beginners and students. The system ensures efficient data handling, secure user access, and smooth interaction between frontend and backend components.

Features of the proposed system:

- User registration and secure login functionality
- Interactive dashboard for easy navigation
- Stock listings and price details viewing
- Buy and sell functionality (simulated trading)
- Portfolio management and user holdings tracking
- Transaction history for activity monitoring
- Responsive and user-friendly interface design
- Scalable architecture using the MERN stack (MongoDB, Express.js, React.js, Node.js)

3.3 Feasibility Study

3.3.1 Technical Feasibility

Technical feasibility refers to the evaluation of technical resources, tools, and technologies required to develop and implement the proposed stock trading web application. The system is built using the MERN stack (MongoDB, Express.js, React.js, Node.js), which are widely used and well-supported technologies.

These tools provide a strong foundation for developing scalable, efficient, and responsive web applications. The development environment is easy to set up, and sufficient online resources and documentation are available to support the implementation process.

Since the project is a web-based application, it does not require high-end hardware or specialized infrastructure, making it technically feasible for academic purposes. Overall, the availability of modern technologies and development tools ensures smooth execution and system maintenance.

Use of MERN stack ensures efficient full-stack development

- Availability of open-source tools and libraries

- Easy integration between frontend and backend using APIs
- No requirement of high-end hardware or complex infrastructure
- Strong community support and extensive documentation for troubleshooting
- These tools are efficient and suitable for project development.

3.3.2 Economic Feasibility

Economic Feasibility evaluates the cost-effectiveness of the proposed stock trading web application. The project is developed using the MERN stack, which consists of open-source technologies, eliminating the need for expensive software licenses. Development can be carried out using standard computing resources without requiring additional investment in hardware. Since the system is designed for educational and demonstration purposes, there are no costs associated with real financial transactions or third-party integrations like payment gateways.

Use of open-source technologies reduces development cost:

- No need for paid software licenses or tools
- Minimal hardware requirements (standard PC/laptop sufficient)
- No investment required for real trading or financial transactions
- Low maintenance and operational cost.

3.3.3 Operational Feasibility

Operational Feasibility focuses on how effectively the proposed system will function in a real-world or user environment. The application is designed with a simple and user-friendly interface, making it easy for users, especially beginners, to understand and operate. The system provides clear navigation, interactive dashboards, and well-structured features such as stock viewing, trading, and portfolio management.

Since the platform operates in a simulated environment, users can learn and practice without any financial risk. The system does not require specialized training, and users can quickly adapt to its functionality. Overall, the application is practical, efficient, and capable of meeting user requirements effectively.

- Easy to use interface suitable for beginners
- No special training required to operate the system
- Smooth navigation and user-friendly design
- Simulated environment reduces risk and increases usability
- Efficient handling of user activities and data

3.3.4 Time Feasibility

Time feasibility refers to the evaluation of whether the proposed stock trading web application can be developed and completed within the given time frame. The project is planned as an academic assignment and is designed with clearly defined modules such as user authentication, stock management, trading functionality, and portfolio tracking.

Since the application uses the MERN stack, which supports rapid development and easy integration, the implementation process becomes faster and more efficient. Proper planning, task division, and use of available resources ensure that each phase of development—analysis, design, coding, and testing—can be completed on schedule.

Therefore, the project is considered time feasible as it can be successfully developed within the allotted duration

- Project can be completed within the academic timeline
- Use of MERN stack enables faster development
- Clear division of modules simplifies implementation
- Availability of resources and tools reduces development time
- Proper planning ensures timely testing and deployment.

3.4 Requirement Analysis

3.4.1 Functional Requirements

Functional requirements describe the core functionalities and operations that the proposed stock trading web application must perform. These requirements define how the system behaves and what features it provides to the users.

The application is designed to allow users to interact with the system efficiently and perform essential trading activities within a simulated environment.

The following are the key functional requirements of the system:

- **User Registration:** The system should allow new users to create an account by providing necessary details such as name, email, and password.
- **User Login and Authentication:** The system should provide secure login functionality and verify user credentials before granting access.
- **User Logout:** Users should be able to log out securely from their accounts.
- **View Stock Listings:** The system should display a list of available stocks along with their price and basic details.
- **Buy Stocks:** The system should allow users to purchase stocks in a simulated environment.
- **Sell Stocks:** Users should be able to sell their owned stocks easily.
- **Portfolio Management:** The system should maintain and display the user's portfolio, including holdings and balance.
- **Dashboard Display:** The system should provide an interactive dashboard showing relevant data and summaries.
- **Data Validation:** The system should validate user inputs to ensure accuracy and prevent errors.
- **Error Handling:** The system should handle errors gracefully and display appropriate messages to users.

3.4.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance standards of the proposed stock trading web application. These requirements do not describe specific functionalities

but focus on how the system performs, behaves, and operates under different conditions. They ensure that the application is efficient, secure, reliable, and user-friendly.

The following are the key non-functional requirements of the system:

Performance: The system should respond quickly to user requests and handle multiple operations efficiently without significant delay.

Scalability: The application should be able to handle an increasing number of users and data without affecting performance.

Security: Proper authentication and authorization mechanisms should be implemented to protect user data and prevent unauthorized access.

Usability: The system should have a simple, intuitive, and user-friendly interface so that users can easily navigate and perform actions.

Reliability: The application should function consistently without crashes or failures and should provide accurate results.

Availability: The system should be accessible to users whenever required, with minimal downtime.

Maintainability: The system should be easy to maintain, update, and modify as per future requirements.

Compatibility: The application should work smoothly across different devices, browsers, and operating systems.

Data Integrity: The system should ensure that data is stored and processed accurately without loss or corruption.

Response Time: The application should process user actions (like login, buy/sell) within an acceptable time limit.

3.5 System Design

The system design of the Zerodha Clone, inspired by Zerodha, defines the overall structure and functioning of the stock trading application. It follows a client-server architecture that divides the system into three main components: frontend, backend, and database.

The frontend, developed using React.js, provides a user-friendly interface where users can view stock prices, place buy and sell orders, and manage their portfolios.

The backend, built with Node.js and Express.js, handles business logic, processes user requests, and ensures proper communication between the frontend and the database.

The database, typically implemented using MongoDB, stores essential data such as user details, stock information, transaction records, and portfolio data. The system operates by sending requests from the user interface to the backend, which processes the data and interacts with the database before returning a response to the user.

3.6 Architecture Design

The architecture of the proposed stock trading web application is based on a client-server model using the MERN stack, which includes MongoDB, Express.js, React.js, and Node.js.

In this architecture, the frontend (client side) is developed using React.js, which provides an interactive and responsive user interface. Users interact with the application through the browser, where they can perform actions such as login, view stock data, and execute trading operations. The frontend communicates with the backend through RESTful APIs, ensuring smooth data exchange and user interaction.

The backend (server side) is developed using Node.js and Express.js, which handle business logic, request processing, and API management. When a user performs any action, such as buying or selling stocks, the request is sent to the server, where it is processed and validated. The server then interacts with the MongoDB database to store or retrieve data, such as user details, stock information, and transaction records. MongoDB, being a NoSQL database, provides flexibility and efficient data handling for dynamic and large datasets.

The architecture is designed to be modular, scalable, and efficient. Each component of the system—frontend, backend, and database—operates independently but communicates seamlessly through APIs. This separation of concerns improves maintainability and allows future enhancements, such as real-time data integration or mobile application support.

Security measures such as authentication and data validation are incorporated to ensure safe and reliable system operations. Overall, the architecture design provides a strong foundation for building a robust, user-friendly, and scalable stock trading application.

3.6.1. Presentation Layer

The Presentation Layer represents the front-end part of the stock trading web application, which is directly accessed by the users. It is responsible for displaying information, capturing user inputs, and providing an interactive interface for a smooth user experience. In this project, the presentation layer is developed using React.js, which allows the creation of dynamic, responsive, and reusable UI components. The interface is designed to be simple and user-friendly, so that even beginners can easily navigate through the application.

This layer includes various user interface components such as login and registration forms, dashboard, stock listings, portfolio view, and transaction history pages. It ensures that all data received from the backend is presented in a clear and structured manner. Interactive elements like buttons, forms, and charts are used to enhance usability and improve user engagement. The presentation layer also handles basic input validation before sending data to the server, reducing errors and improving system efficiency.

Additionally, the presentation layer communicates with the backend through RESTful APIs using HTTP requests. It sends user actions (such as login, buy/sell requests) to the server and displays the response received from it.

Components:

Login Page: Provides a login form for user authentication.

Registration Page: Provides the facility for new users to create an account.

Dashboard: Gives the user an overview (stocks, portfolio summary, etc.)

Stock Listing Page: Displays available stocks and their details

Search & Filter Component: Helps users find specific stocks

Buy/Sell Interface: Allows users to perform trading operations

Portfolio View: Shows user's holdings and investment details

Transaction History Page: Displays past trading records

Navigation Bar (Navbar): Provides easy navigation between different sections

Charts & Graphs: Used for stock trends and data visualization; provides a login form for user authentication

Registration Page: Allows new users to create an account

Dashboard: Provides the user with an overview (stocks, portfolio summary, etc.)

Stock Listing Page: Displays available stocks and their details

Search & Filter Component: Helps users find specific stocks

Buy/Sell Interface: Allows users to perform trading operations

Portfolio View: Shows the user's holdings and investment details

Transaction History Page: Displays past trading records

Navigation Bar (Navbar): Provides easy navigation between different sections

Charts & Graphs: Used for stock trends and data visualization

3.6.2. Application Layer

The Application Layer represents the backend part of the stock trading web application, where all the core logic and processing take place. It acts as an intermediary between the presentation layer and the database layer. This layer is developed using Node.js and Express.js, which handle server-side operations, request processing, and API management.

When a user performs any action such as login, buying or selling stocks, the request is sent to this layer, where it is validated and processed according to the defined business logic. The application layer ensures proper communication between the frontend and the database by using RESTful APIs.

This layer is responsible for implementing important functionalities such as user authentication, transaction management, portfolio updates, and error handling. It also ensures data security by validating inputs and preventing unauthorized access.

The application layer processes all incoming and outgoing data efficiently, ensuring that the system performs smoothly and reliably. Overall, it plays a crucial role in maintaining system functionality, performance, and security.

Components:

- API Controller: Handles user requests and returns responses
- Authentication Module: Manages user login, registration, and security
- Business Logic Module: Implements buy/sell operations and system rules
- Transaction Handler: Processes and manages trading transactions
- Portfolio Manager: Updates and maintains user portfolios
- Validation Module: Verifies input data and prevents errors
- Error Handling Module: Handles errors and displays proper messages
- API Routes: Manages communication between frontend and backend.

3.6.3. Database Layer

The Database Layer is responsible for storing, managing, and retrieving data used in the stock trading web application. In this project, MongoDB is used as the database, which is known for its flexibility and scalability. This layer stores important information such as user details, stock data, transaction records, and portfolio information. The database interacts with the application layer to perform operations like data insertion, updating, deletion, and retrieval based on user requests.

The use of MongoDB allows the system to handle large amounts of dynamic data efficiently. Data is stored in the form of collections and documents, which makes it easier to manage and modify as per application requirements. The database layer ensures data consistency, integrity, and security by implementing proper schema design and validation techniques.

It also supports fast data access, which improves the overall system performance. Overall, the database layer plays a critical role in maintaining reliable and organized data storage for the application.

Components:

- User Collection: The user stores details (name, email, password)
- Stock Collection: It stores stock information and price data.

- Transaction Collection: It maintains buy/sell records and history.
- Portfolio Collection: User ke holdings aur investment data store karta hai
- Database Schema: Defines the data structure
- CRUD Operations: Performs Create, Read, Update, Delete operations.
- Data Validation: Ensures data accuracy and consistency.
- Database Connection Module: Manages the connection between the backend and the database

3.7. Module Design

The module design of the proposed stock trading web application follows a structured and modular approach to ensure maintainability, scalability, and efficient system performance. The system is divided into multiple functional modules, each responsible for a specific task. This modularization helps in reducing system complexity and allows independent development and testing of each component.

The modules interact with each other through well-defined interfaces, ensuring smooth data flow and communication across the system. Each module is designed to handle a particular aspect of the application, such as user authentication, stock data management, trading operations, and portfolio tracking.

3.7.1. User Module

This module handles user-related functionalities such as registration, login, and logout. It ensures secure authentication and authorization of users by validating credentials and managing user sessions.

3.7.2. Stock Management Module

This module is responsible for managing and displaying stock-related information. It provides users with stock listings, price details, and other relevant data required for trading decisions.

3.7.3. Trading Module

The trading module enables users to perform buy and sell operations in a simulated environment. It processes user requests and updates the system accordingly, ensuring proper transaction handling and system updates.

3.7.4. Portfolio Management Module

This module maintains and updates the user's portfolio based on trading activities. It displays holdings, investment details, and overall portfolio performance.

3.7.5 Dashboard Module

This module provides a summary view of user data, including stock information, portfolio status, and recent transactions through an interactive interface.

3.8 Database Design

User Collection

- ❏ User ID
- ❏ Name
- ❏ Email
- ❏ Password

Stock Collection

- ❏ Stock ID
- ❏ Stock Name
- ❏ Stock Symbol
- ❏ Price
- ❏ Market Data

Portfolio Collection

- ❏ Portfolio ID

- ❑ User ID
- ❑ Stock ID
- ❑ Quantity Owned
- ❑ Total Investment

3.9 Process Flow of System

The process flow of the proposed stock trading web application describes the sequence of operations performed by the system from user interaction to data processing and output generation. The system follows a structured flow where the user initiates actions through the presentation layer, which are then processed by the application layer and stored or retrieved from the database layer.

Initially, the user accesses the application through the web interface and performs registration or login. Once authenticated, the user is redirected to the dashboard, where they can view stock listings, portfolio details, and other relevant information. When the user performs any action such as buying or selling stocks, the request is sent to the backend server. The application layer processes the request, validates input, and interacts with the database to update or retrieve data accordingly.

After processing the request, the server sends the response back to the frontend, where the updated information is displayed to the user. The system continuously maintains records of transactions and updates the portfolio in real time (or simulated real time). This structured flow ensures that all operations are executed efficiently while maintaining data consistency and system reliability.

Steps in Processes Flow

- User opens the application through a web browser
- User registers a new account or logs into an existing account
- System authenticates user credentials
- User is redirected to the dashboard
- User views stock listings and details
- User performs buy or sell operations

- Request is sent to the backend server
- Server validates input and processes the request
- Database is updated with transaction and portfolio data
- Response is sent back to the frontend
- Updated information is displayed to the user
- Transaction history and portfolio are maintained

3.9.1 Data Flow Diagram

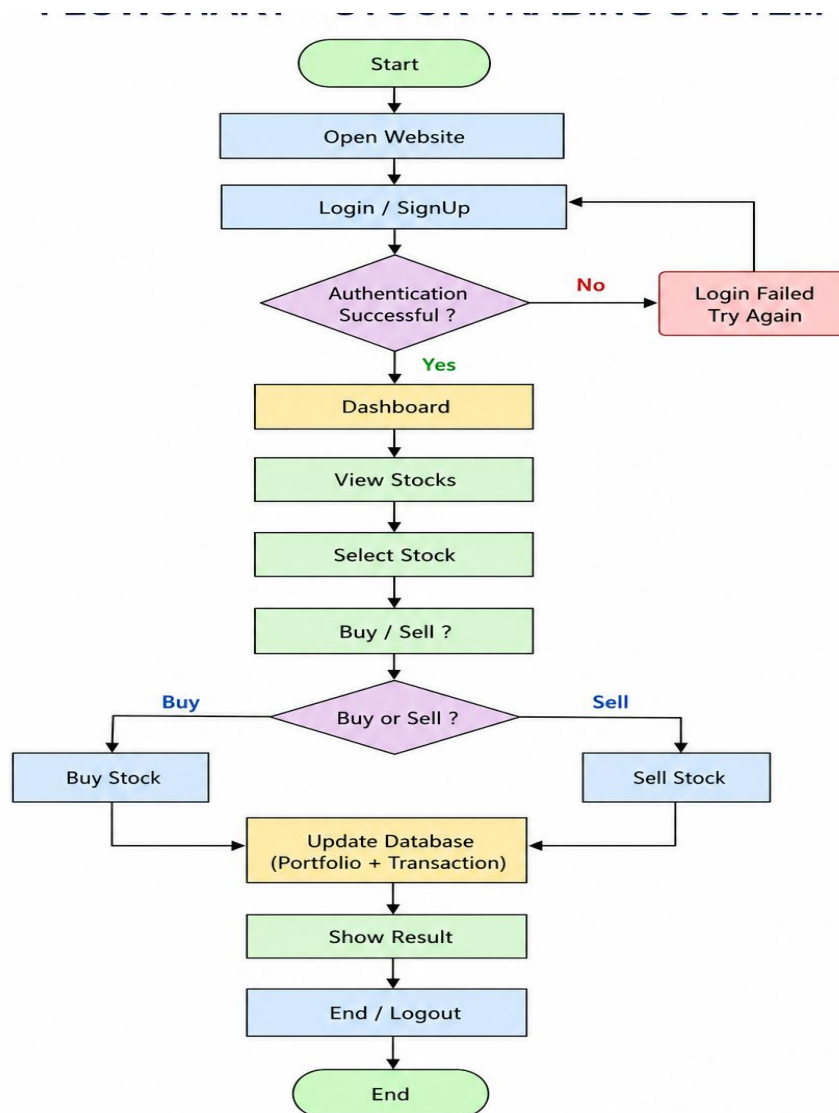


Figure: 3.3.1 Data Flow Diagram

Level 0 & Level 1 DFD

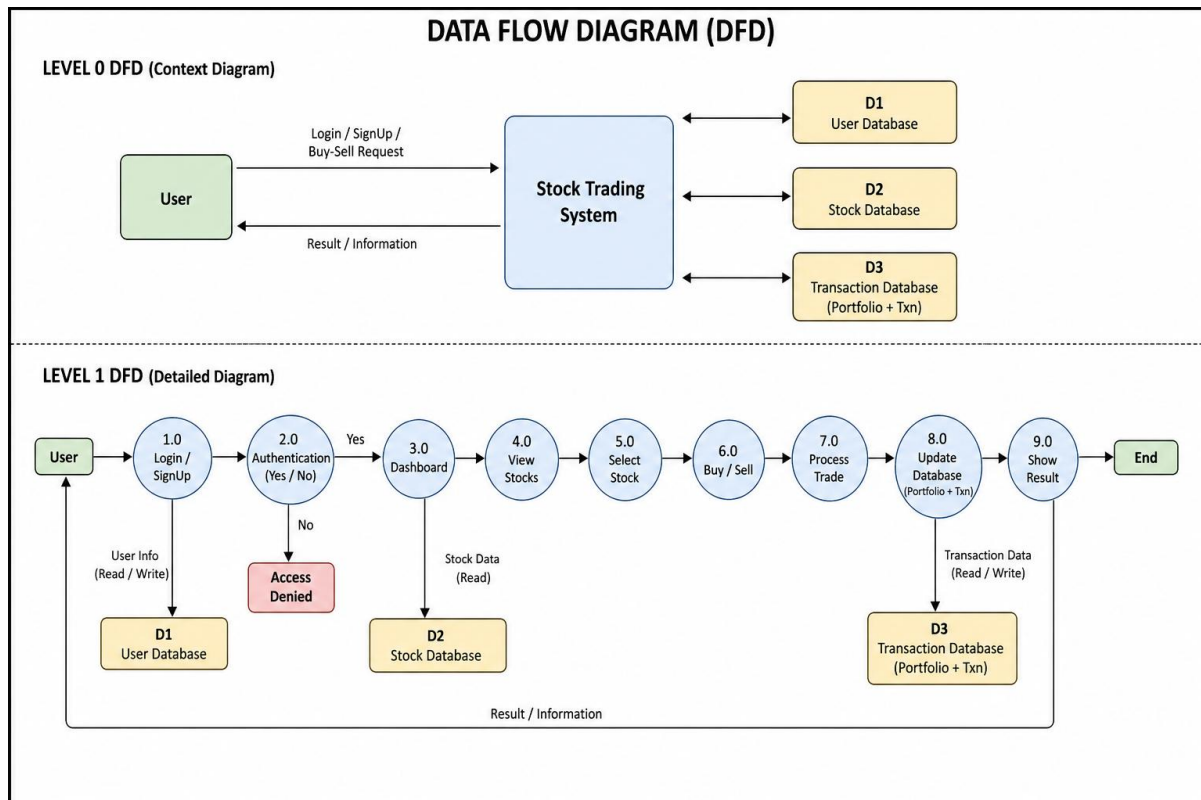


Figure: 3.3.2 Level 0 & Level 1 DFD

3.9.2. Use Case Diagram (Text Form)

Actors:

- ❑ User
- ❑ Admin

User Actions:

- ❑ Register
- ❑ Login
- ❑ View Result
- ❑ View History

Admin Actions:

❑ Manage Users

❑ Update Disease Data

3.9.3. Interface Design

The user interface is simple, modern, and responsive. .

Main features:

❑ Clean homepage

❑ Easy navigation menu

❑ Upload button

❑ Result cards

❑ Mobile-friendly layout

❑ Attractive dashboard

3.9.4. Security Design

Security measures included in the system:

- User Authentication: Secure login using email and password
- Authorization: Role-based or session-based access control
- Password Encryption: Passwords are stored in encrypted/hashed form
- Input Validation: Prevents invalid or malicious data entry
- Secure API Communication: Use of HTTP/HTTPS protocols
- Error Handling: Prevents exposure of sensitive system information
- Data Protection: Secure storage and controlled database access

3.10. Testing Strategy

The system is designed for testing through:

- Unit Testing: Testing individual modules or components independently
- Integration Testing: Testing the interaction between different modules
- System Testing: Testing the complete system as a whole
- User Acceptance Testing (UAT): Testing the system from the end-user perspective
- Performance Testing: Evaluating system speed, scalability, and response time
- Security Testing: Verifying authentication, authorization, and data protection mechanisms

3.11. Advantages of Designed System

- User-Friendly Interface: Simple and intuitive design makes it easy for beginners to use
- Simulated Trading Environment: Allows users to practice trading without real financial risk
- Full-Stack Implementation: Demonstrates complete MERN stack development (frontend + backend + database)
- Efficient Data Management: Uses MongoDB for flexible and scalable data handling
- Secure System: Implements authentication and data protection mechanisms
- Real-Time/Interactive Experience: Provides dynamic updates and responsive UI
- Modular Design: Easy to maintain, update, and extend with new features
- Educational Value: Helps students understand real-world trading systems practically
- Cross-Platform Compatibility: Works on different devices and browsers
- Scalability: Can be enhanced with advanced features in the future

3.12 Future Enhancements

- Real-Time Stock Data Integration: Integrate live stock market APIs to display real-time prices and updates
- Payment Gateway Integration: Enable real money transactions for actual trading functionality

- **Advanced Charts and Analytics:** Add technical indicators, graphs, and data analysis tools
- **Mobile Application Development:** Develop Android/iOS apps for better accessibility
- **Notification System:** Provide alerts for stock price changes, transactions, and updates
- **AI-Based Recommendations:** Suggest stocks based on user behavior and market trends
- **Multi-User Roles:** Introduce roles such as admin, trader, and analyst with different permissions
- **Enhanced Security Features:** Implement two-factor authentication (2FA) and advanced encryption
- **Cloud Deployment:** Host the application on cloud platforms for better scalability and availability
- **Portfolio Analytics Dashboard:** Provide detailed insights, profit/loss analysis, and performance reports.

CHAPTER 4: IMPLEMENTATIONS

The implementation phase of the proposed stock trading web application involves the actual development of the system based on the design specifications. This phase focuses on converting the system design into a working application using the MERN stack, which includes MongoDB, Express.js, React.js, and Node.js. The development process is divided into frontend and backend implementation, ensuring proper integration and smooth functionality of the system.

On the frontend, React.js is used to create an interactive and responsive user interface. Various components such as login, registration, dashboard, stock listing, portfolio, and transaction history are developed using reusable components. The frontend communicates with the backend through RESTful APIs using HTTP requests. Proper state management and routing are implemented to ensure smooth navigation and dynamic updates of the user interface.

4.1. Implementation

4.1.1 Frontend Implementation

The frontend of the application is developed using React.js, which enables the creation of a dynamic and responsive user interface. The interface is designed with a focus on simplicity and ease of use so that users can interact with the system without any difficulty.

- Developed using React.js for dynamic UI
- Component-based architecture for reusability
- Responsive design for multiple devices
- Navigation using React Router
- API integration with backend services
- Real-time or dynamic data updates
- Form validation and error handling
- Interactive dashboard with charts and visuals

The frontend communicates with the backend using API calls, ensuring seamless data exchange.

4.1.2. Backend Implementation

The backend implementation of the proposed stock trading web application is responsible for handling server-side logic, data processing, and communication between the frontend and the database. The backend is developed using Node.js and Express.js, which provide a robust and scalable environment for building RESTful APIs. These APIs handle client requests, process business logic, and send appropriate responses back to the frontend.

- Developed using Node.js and Express.js
- RESTful API development for client-server communication
- Integration with MongoDB database
- User authentication using JWT
- Password hashing for security
- Routing and middleware handling
- Input validation and error handling
- Efficient data processing and business logic implementation
- This layer ensures efficient processing of user data and provides accurate responses.

4.1.3. Database Implementation

The database implementation of the proposed stock trading web application is carried out using MongoDB, a NoSQL database that provides flexibility and scalability for handling dynamic data. The database is used to store and manage all essential information such as user details, stock data, transaction records, and portfolio information.

MongoDB stores data in the form of collections and documents, which allows easy modification and efficient data handling.

- Use of MongoDB for flexible and scalable data storage
- Mongoose for schema definition and data modeling
- Implementation of CRUD operations

- Efficient data retrieval using indexing
- Use of references to maintain relationships
- Data validation to ensure accuracy and consistency
- Secure database connection and access control
- Optimized performance for handling large datasets

The database plays a vital role in maintaining system reliability and performance.

4.1.4. AI Model Implementation

Artificial Intelligence is integrated into the system to provide intelligent analysis of plant conditions

- ☐ The AI model processes input data provided by users
- ☐ It identifies possible plant diseases based on symptoms or input details
- ☐ It generates recommendations such as watering frequency and sunlight requirements
- ☐ Machine Learning techniques improve the accuracy of predictions over time

This component adds intelligence to the system and enhances decision-making capabilities.

4.1.5. Chatbot Implementation

The chatbot is implemented as a key feature of the application to provide interactive support.

- ☐ Built using Natural Language Processing (NLP) techniques
- ☐ Understands user queries related to plant care
- ☐ Generates appropriate responses in real time
- ☐ Integrated into the frontend for easy accessibility

The chatbot improves user engagement and provides instant assistance.

4.2. API Integration

API Integration plays a crucial role in the proposed stock trading web application by enabling communication between the frontend and backend systems. The application uses RESTful APIs to send and receive data between the client side (React.js) and server side (Node.js and Express.js).

These APIs handle various operations such as user authentication, fetching stock data, executing buy and sell transactions, and updating portfolio details. The use of APIs ensures a structured and efficient way of exchanging data across different components of the system.

APIs are used to connect different components of the system:

- Use of RESTful APIs for communication
- HTTP methods (GET, POST, PUT, DELETE) for operations
- JSON format for data exchange
- Secure API access using authentication tokens (JWT)
- Error handling and response management
- Integration between frontend, backend, and database
- Scalable and efficient request handling
- Smooth real-time or dynamic data updates

4.3. Testing and Debugging

Testing and debugging are essential phases in the development of the proposed stock trading web application, ensuring that the system functions correctly, efficiently, and without errors.

Testing involves verifying that all modules of the application perform as expected, while debugging focuses on identifying and fixing errors or bugs encountered during development. These processes help improve the quality, reliability, and performance of the system before deployment.

After implementation, the system undergoes testing to ensure proper functionality.

- Writing and executing test cases for all modules

- Identifying and fixing bugs in frontend and backend
- Validating user inputs and system responses
- Testing API endpoints and database operations
- Using debugging tools (console, logs, browser dev tools)
- Ensuring proper error handling and messages
- Checking system performance and response time
- Verifying overall system stability and reliability

4.4. Deployment

Deployment is the final phase of the development process, where the stock trading web application is made available for users to access over the internet. This phase involves hosting the frontend, backend, and database on suitable platforms to ensure that the application runs smoothly in a real-world environment.

The deployment process ensures that all components of the system are properly configured, integrated, and accessible to end users.

- Creating a Render account and setting up a new web service
- Connecting GitHub repository to Render
- Deploying backend (Node.js + Express) on Render
- Configuring environment variables (DB URL, JWT secret)
- Connecting MongoDB Atlas database
- Deploying frontend or integrating with backend
- Testing APIs and application in live environment
- Monitoring logs and fixing deployment errors

This allows users to access the system from anywhere with an internet connection.

CHAPTER 5: SOURCE CODE

(I)

```
import React from "react";

import { useForm } from "react-hook-form";

import { yupResolver } from "@hookform/resolvers/yup";

import * as yup from "yup";

import axios from 'axios';

import swal from 'sweetalert2'

import { useNavigate } from "react-router-dom";

import { Link } from "react-router-dom";

function Signin() {

  const nav= useNavigate();

  const schema = yup

    .object()

    .shape({

      email: yup.string().required().matches(/^[^\s@]+@[^\s]+\.(com|in|org|net)$/, "Enter valid email"),

      password: yup.string().required().min(5).max(20),

    })

    .required();

  const { register, handleSubmit,formState: {errors} } = useForm({

    resolver: yupResolver(schema),
```

```

});

const handleRegister=async(data)=>{

  const res=await axios.post("http://localhost:3000/login",data);

  if(res.data.success===true){

    swal.fire({

      title:"Login",

      text:res.data.message,

      icon:"success"

    });

    localStorage.setItem("token",res.data.token);

    window.dispatchEvent(new Event("authChange"));

    window.location.href="http://localhost:5173"

  }else{

    swal.fire({

      title:"Not Login",

      text:res.data.message,

      icon:"error"

    })

  }

}

return (

  <

  <div className="row">

```

```
<div className="col-sm-4"></div>
```

```
<div className="col-sm-4 mt-4" style={{boxShadow:"0 5px 15px",borderRadius:"10px"}}>
```

```
<form onSubmit={handleSubmit(handleRegister)}>
```

```
<h1 className="text-primary text-center">Login here</h1>
```

```
<hr/>
```

```
<input type="email" style={{border:"none",outline:"none",borderBottom:"2px dotted black"}} {...register('email')} className="form-control mb-3" placeholder="Enter Your Email"></input>
```

```
{errors?.email && <p className="text-danger">{errors?.email?.message}</p>}
```

```
<input type="password" style={{border:"none",outline:"none",borderBottom:"2px dotted black"}} {...register('password')} className="form-control mb-3" placeholder="Enter Your Password"></input>
```

```
{errors?.password && <p className="text-danger">{errors?.password?.message}</p>}
```

```
<input
```

```
type="submit"
```

```
className="form-control mb-3 bg-primary text-light"
```

```
value="Login"
```

```
></input>
```

```
</form>
```

```
<p className="text-center" style={{textDecoration:"none"}}>Don't have an account <Link to="/signup" className="login">Signup</Link></p>
```

```
</div>
```

```
<div className="col-sm-4"></div>
```

```
</div>
```

</>

);

}

export default Signin;

(II)

import React from "react";

import { useForm } from "react-hook-form";

import { yupResolver } from "@hookform/resolvers/yup";

import * as yup from "yup";

import axios from 'axios';

import swal from 'sweetalert2';

import { useNavigate } from "react-router-dom";

import { Link } from "react-router-dom";

function Signup() {

const nav= useNavigate()

const schema = yup

.object()

.shape({

name: yup.string().required().min(3).max(30),

email: yup.string().required().matches(/^[^\s@]+@[^\s]+\.(com|in|org|net)\$/, "Enter valid email"),

password: yup.string().required().min(5).max(20),

})


```

.required();

const { register, handleSubmit,formState:{errors} } = useForm({

  resolver: yupResolver(schema),

});

const handleRegister=async(data)=>{

  const res=await axios.post("http://localhost:3000/register",data);

  if(res.data.success===true){

    swal.fire({

      title:"Register",

      text:res.data.message,

      icon:"success"

    });

    nav("/signin")

  }else{

    swal.fire({

      title:"Not Register",

      text:res.data.message,

      icon:"error"

    })

  }

}

return (

  <

```

```

<div className="row">

  <div className="col-sm-4"></div>

  <div      className="col-sm-4      mt-4      "      style={{boxShadow:"0      5px
15px",borderRadius:"10px"}}>

    <form onSubmit={handleSubmit(handlleRegister)}>

      <h2 className="text-primary text-center">Register here</h2>

      <hr/>

      <input type="text" style={{border:"none",outline:"none",borderBottom:"2px solid black"}}
{...register('name')}} className="form-control mb-3" placeholder="Enter Your Name"></input>

      {errors?.name && <p className="text-danger">{errors?.name?.message}</p>}}

      <input  type="email"  style={{border:"none",outline:"none",borderBottom:"2px  solid
black"}}  {...register('email')}}  className="form-control  mb-3"  placeholder="Enter  Your
Email"></input>

      {errors?.email && <p className="text-danger">{errors?.email?.message}</p>}}

      <input  type="password"  style={{border:"none",outline:"none",borderBottom:"2px  solid
black"}}  {...register('password')}}  className="form-control  mb-3"  placeholder="Enter  Your
Password"></input>

      {errors?.password && <p className="text-danger">{errors?.password?.message}</p>}}

      <input

        type="submit"

        className="form-control mb-3 bg-primary text-light"

        value="Register"

      ></input>

    </form>

```

```
    <p className="text-center" style={{textDecoration:"none"}}>User already have an  
account <Link to="/signin" className="login">Signin</Link></p>
```

```
</div>
```

```
<div className="col-sm-4"></div>
```

```
</div>
```

```
</>
```

```
);
```

```
}
```

```
export default Signup;
```

(III)

```
import React, { useEffect, useState } from "react";
```

```
import { Link, useLocation, useNavigate } from "react-router-dom";
```

```
import { FaInfoCircle } from "react-icons/fa";
```

```
import { RiMoneyRupeeCircleFill } from "react-icons/ri";
```

```
import { AiFillProduct } from "react-icons/ai";
```

```
import { HiSupport } from "react-icons/hi";
```

```
import { GiArchiveRegister } from "react-icons/gi";
```

```
import { SiSimplelogin } from "react-icons/si";
```

```
import { LuLogOut } from "react-icons/lu";
```

```
import { FaChartLine } from "react-icons/fa";
```

```
import { FaPhoneVolume } from "react-icons/fa6";
```

```
function Navbar() {
```

```
    const [token, settoken] = useState(localStorage.getItem("token"));
```

```

const nav = useNavigate();

const location = useLocation();

useEffect(()=>{

  const syncAuth=()=>{

    settoken(localStorage.getItem("token"))

  };

  window.addEventListener("authChange",syncAuth);

  return ()=>{

    window.removeEventListener("authChange",syncAuth);

  };

},[])

const handlelogout = () => {

  localStorage.removeItem("token");

  window.dispatchEvent(new Event("authChange"));

  nav("/");

};

return (

  <nav

    className="navbar navbar-expand-lg border-bottom"

    style={{ backgroundColor: "#FFF" }}

  >

    <div className="container P-2">

      <Link className="navbar-brand" to="/">

```

```

</img>

</Link>

<button
  className="navbar-toggler"
  type="button"
  data-bs-toggle="collapse"
  data-bs-target="#navbarSupportedContent"
  aria-controls="navbarSupportedContent"
  aria-expanded="false"
  aria-label="Toggle navigation"
>

  <span className="navbar-toggler-icon"></span>

</button>

<div className="collapse navbar-collapse" id="navbarSupportedContent">
  <form className="d-flex" role="search">
    <ul className="navbar-nav me-auto mb-2 mb-lg-0">
      <li className="nav-item">
        <Link className="nav-link active d-flex align-items-center gap-2" to="/about">
          <FaInfoCircle/>

```

About

</Link>

<li className="nav-item">

<Link className="nav-link active d-flex align-items-center gap-2" to="/product">

<AiFillProduct/>

Product

</Link>

<li className="nav-item">

<Link className="nav-link active d-flex align-items-center gap-2" to="/pricing">

<RiMoneyRupeeCircleFill/>

Pricing

</Link>

<li className="nav-item">

<Link className="nav-link active d-flex align-items-center gap-2" to="/support">

<HiSupport/>

Support

</Link>

<li className="nav-item">

<Link className="nav-link active d-flex align-items-center gap-2" to="/contact">

<FaPhoneVolume/>

ContactUs

</Link>

{!token ? (

<

<li className="nav-item">

<Link

className="nav-link active d-flex align-items-center gap-2"

aria-current="page"

to="/Signup"

>

<GiArchiveRegister/>

Signup

</Link>

<li className="nav-item">

<Link className="nav-link active d-flex align-items-center gap-2" to="/signin">

<SiSimplelogin/>

Signin

</Link>

</>

```

): (
  <
  <li className="nav-item">
    <a className="nav-link d-flex align-items-center gap-2" href="http://localhost:5173">
      <FaChartLine/>
      Dashboard
    </a>
  </li>
  <li className="nav-item">
    <button
      className="nav-link active d-flex align-items-center gap-2"
      aria-current="page"
      onClick={handlelogout}
    >
      <LuLogOut/>
      Logout
    </button>
  </li>
</>
)}
</ul>
</form>
</div>

```



```
</div>
```

```
</nav>
```

```
);
```

```
}
```

```
export default Navbar;
```

(IV)

```
import React from 'react';
```

```
function Footer() {
```

```
  return (
```

```
    <footer style={{backgroundColor:"rgb(250, 250, 250)}}>
```

```
      <div className='container border-top mt-5'>
```

```
        <div class="row mt-5">
```

```
          <div className='col'>
```

```
            <img src='media\image\images\logo.svg' style={{width:"50%"}}></img>
```

```
            <p>
```

```
              &copy; 2010 - 2025, Zerodha Broking Ltd. All rights reserved.
```

```
            </p>
```

```
          </div>
```

```
        <div className='col'>
```

```
          <p>Company</p>
```

```
          <a href="" style={{textDecoration:"none"}}>About</a>
```

```
          <br />
```

```
<a href="" style={{textDecoration:"none"}}>Products</a>

<br />

<a href="" style={{textDecoration:"none"}}>Referral programme</a>

<br />

<a href="" style={{textDecoration:"none"}}>Careers</a>

<br />

<a href="" style={{textDecoration:"none"}}>Zerodha.tech</a>

<br />

<a href="" style={{textDecoration:"none"}}>Open source</a>

<br />

<a href="" style={{textDecoration:"none"}}>Press & media</a>

<br />

<a href="" style={{textDecoration:"none"}}>Zerodha Cares (CSR)</a>

<br />

</div>

<div className='col'>

<p>Support</p>

<a href="" style={{textDecoration:"none"}}>Contact us</a>

<br />

<a href="" style={{textDecoration:"none"}}>Support portal</a>

<br />

<a href="" style={{textDecoration:"none"}}>Z-Connect blog</a>

<br />
```

List of charges

Downloads & resources

Videos

Market overview

</div>

<div className='col'>

<p>Account</p>

Open an account

Fund transfer

60 day challeng

</div>

<div className='col'></div>

<div className='col'></div>

</div>

<div className='mt-5 text-muted' style={{fontSize:"14px"}}>

<p>Zerodha Broking Ltd.: Member of NSE, BSE & MCX – SEBI Registration no.: INZ000031633 CDSL/NSDL: Depository services through Zerodha Broking Ltd. – SEBI Registration no.: IN-DP-431-2019 Commodity Trading through Zerodha Commodities Pvt. Ltd. MCX: 46025; NSE-50001 – SEBI Registration no.: INZ000038238 Registered Address: Zerodha Broking Ltd., #153/154, 4th Cross, Dollars Colony, Opp. Clarence Public School, J.P Nagar 4th Phase, Bengaluru - 560078, Karnataka, India. For any complaints pertaining to securities broking please write to complaints@zerodha.com, for DP related to dp@zerodha.com. Please ensure you carefully read the Risk Disclosure Document as prescribed by SEBI | ICF</p>

<p>Procedure to file a complaint on SEBI SCORES: Register on SCORES portal. Mandatory details for filing complaints on SCORES: Name, PAN, Address, Mobile Number, E-mail ID. Benefits: Effective Communication, Speedy redressal of the grievances</p>

<p>Smart Online Dispute Resolution | Grievances Redressal Mechanism</p>

<p>Investments in securities market are subject to market risks; read all the related documents carefully before investing.</p>

<p>Attention investors: 1) Stock brokers can accept securities as margins from clients only by way of pledge in the depository system w.e.f September 01, 2020. 2) Update your e-mail and phone number with your stock broker / depository participant and receive OTP directly from depository on your e-mail and/or mobile number to create pledge. 3) Check your securities / MF / bonds in the consolidated account statement issued by NSDL/CDSL every month.</p>

<p>"Prevent unauthorised transactions in your account. Update your mobile numbers/email IDs with your stock brokers. Receive information of your transactions directly from Exchange on your mobile/email at the end of the day. Issued in the interest of investors. KYC is one time exercise while dealing in securities markets - once KYC is done through a SEBI registered intermediary (broker, DP, Mutual Fund etc.), you need not undergo the same process again when you approach another intermediary." Dear Investor, if you are subscribing to an IPO, there is no need to issue a cheque. Please write the Bank account number and sign the IPO application form to authorize your bank to make payment in case of allotment. In case of non allotment the funds will remain in your bank account. As a business we don't give stock tips, and have not authorized anyone to trade on behalf of others. If you find anyone claiming to be part of Zerodha and offering such services, please create a ticket here.</p>

```
</div>
```

```
</div>
```

```
</footer>
```

```
);
```

```
}
```

```
export default Footer;
```

(V)

```
require('dotenv').config();
```

```
const express=require("express");
```

```
const mongoose=require("mongoose");
```

```
const bodyParser = require("body-parser");
```

```
const cors = require("cors");
```

```
const {HoldingsModel} = require("./model/HoldingsModel");
```

```
const {PositionsModel} = require("./model/PositionsModel");
```

```
const {OrdersModel} = require("./model/OrdersModel");
```

```
const {userRegister}=require("../backend/model/userModel");
```

```
const bcrypt=require("bcrypt");
```

```
const jwt=require("jsonwebtoken");
```

```
const stockRoutes=require("./routes/routes.js");
```

```
const PORT = process.env.PORT || 3000;
```

```
const url = process.env.MONGO_URL;
```

```
const app=express();
```

```

app.use(express.json())

app.use(cors());

app.use(bodyParser.json());

app.use("/api",stockRoutes)

app.get("/allHoldings",async(req,res)=>{

    let allHoldings=await HoldingsModel.find({});

    res.json(allHoldings);

})

app.get("/allPositions",async(req,res)=>{

    let allPositions=await PositionsModel.find({});

    res.json(allPositions);

});

app.post("/newOrder",async(req,res)=>{

    let newOrder=new OrdersModel({

        name:req.body.name,

        qty:req.body.qty,

        price:req.body.price,

        mode:req.body.mode,

    });

    newOrder.save();

    res.send("order saved");

});

app.post("/register",async(req,res)=>{

```

```

try{

const {name,email,password}= req.body;

const isExist=await userRegister.findOne({email});

if(isExist){

  res.json({

    success:false,

    code:400,

    message:"User Already Register",

    data:isExist,

    error:true

  });

}else{

  const haspassword=await bcrypt.hash(password,10);

  const data= new userRegister( {name,email,password:haspassword})

  const result=data.save();

  res.json({

    success:true,

    code:200,

    message:"User Register Success",

    data:result,

    error:true

  });

}

```

```

} catch(err){

  res.json({

    success:false,

    code:500,

    message:"Internal Server Error",

    data:[],

    error:true

  });

};

});

app.post("/login",async(req,res)=>{

  try{

    const {email,password}= req.body;

    const data=await userRegister.findOne( {email});

    if(data){

      const isMatch= bcrypt.compare(password,data.password)

      if(isMatch){

        const payload={email:data.email};

        const token=jwt.sign(payload,"Secrete_key",{expiresIn:"1h"});

        res.json({

          success:true,

          code:200,

          message:"Login Success",

```



```
        data:data,

        token,

        error:false

    });

} else {

    res.json({

        success:false,

        code:400,

        message:"Login Faield",

        data:[],

        error:false

    })

}

} else {

    res.json({

        success:false,

        code:404,

        message:"Credential failed",

        data:[],

        error:true

    })

}

} catch(err){
```

```

    res.json({
      success:false,
      code:500,
      message:"Internal Server Error",
      data:[],
      error:true
    })
  }
})

app.listen(PORT,()=>{
  console.log("App started ");
  mongoose.connect(url);
  console.log("DB connected suuccessfully");
});

```

(VI)

```

const axios = require("axios");

const getStock = async (req, res) => {
  try {
    const holdings = [
      { name: "TCS.BSE", qty: 1, avg: 3000 },
      { name: "INFY.BSE", qty: 2, avg: 1400 },
    ];

```

```

const result = await Promise.all(

  holdings.map(async (item) => {

    const response = await axios.get(

      `https://www.alphavantage.co/query?function=GLOBAL_QUOTE&symbol=${item.name}&apikey=D5RJCTV5FB3OFRT4`

    );

    const data = response.data["Global Quote"];

    if (!data || !data["05. price"]) {

      return {

        name: item.name,

        qty: item.qty,

        avg: item.avg,

        price: "N/A",

        net: "N/A",

        day: "N/A",

      };

    }

    const price = parseFloat(data["05. price"]);

    const net = ((price - item.avg) / item.avg) * 100;

    return {

      name: item.name,

      qty: item.qty,

```

```

    avg: item.avg,

    price: price.toFixed(2),

    net: net.toFixed(2) + "%",

    day: data["10. change percent"],

  };

})

);

res.json(result);

} catch (err) {

  console.log(err.message);

  res.status(500).json({ message: err.message });

}

};

module.exports = { getStock };

```

(VII)

```

import { useEffect, useState } from 'react'

import reactLogo from './assets/react.svg'

import viteLogo from '/vite.svg'

import './App.css'

function App() {

  const [count, setCount] = useState(0);

  useEffect(()=>{

```

```

const token=localStorage.getItem("token");

if(token){

  window.location.href='http://localhost:5173/signin'

}

},[])

return (

  <

  <div>

    <a href="https://vite.dev" target="_blank">

      <img src={viteLogo} className="logo" alt="Vite logo" />

    </a>

    <a href="https://react.dev" target="_blank">

      <img src={reactLogo} className="logo react" alt="React logo" />

    </a>

  </div>

  <h1>Vite + React</h1>

  <div className="card">

    <button onClick={() => setCount((count) => count + 1)}>

      count is {count}

    </button>

    <p>

      Edit <code>src/App.jsx</code> and save to test HMR

    </p>

```

```
</div>
```

```
<p className="read-the-docs">
```

Click on the Vite and React logos to learn more

```
</p>
```

```
</>
```

```
)
```

```
}
```

```
export default App
```

CHAPTER 6: RESULTS & DISCUSSIONS

6.1. Results

The proposed stock trading web application was successfully designed, developed, and deployed using modern web technologies. The system was tested in a simulated environment, and the results demonstrate that all major functionalities are working as expected. Users are able to register, log in securely, view stock listings, perform buy and sell operations, and manage their portfolio efficiently. The integration between the frontend, backend, and database ensures smooth data flow and real-time (or near real-time) updates within the application.

During testing, the application showed stable performance under normal usage conditions. The response time for user actions such as login, stock selection, and trading operations was found to be satisfactory. The modular architecture of the system contributed to easy debugging and maintenance. The use of the MERN stack enabled efficient data handling and seamless communication between different layers of the application. Additionally, deployment on the Render platform ensured that the application is accessible online and can handle user requests reliably.

The discussion of results highlights that the system meets both functional and non-functional requirements defined during the analysis phase. The user interface is simple and intuitive, making it easy for beginners to navigate and perform operations. The simulated trading environment allows users to practice without financial risk, fulfilling the educational objective of the project. Security features such as authentication and data validation are implemented effectively, ensuring safe system usage.

Overall, the results indicate that the system performs efficiently and achieves its intended objectives. The project demonstrates the practical application of full-stack development concepts and provides a strong foundation for building more advanced trading platforms in the future.

- Successful implementation of all core features
- Smooth integration of frontend, backend, and database
- User-friendly interface and easy navigation
- Reliable system performance and response time

- Secure authentication and data handling
- Successful deployment on Render platform

List of Figures

6.1.1 Home Page

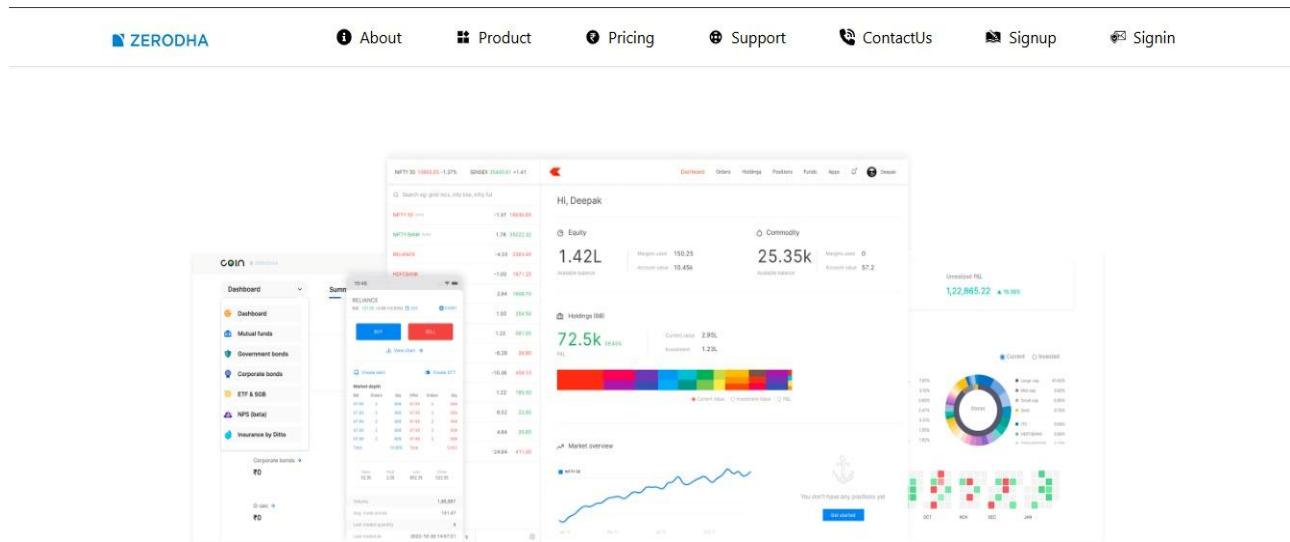


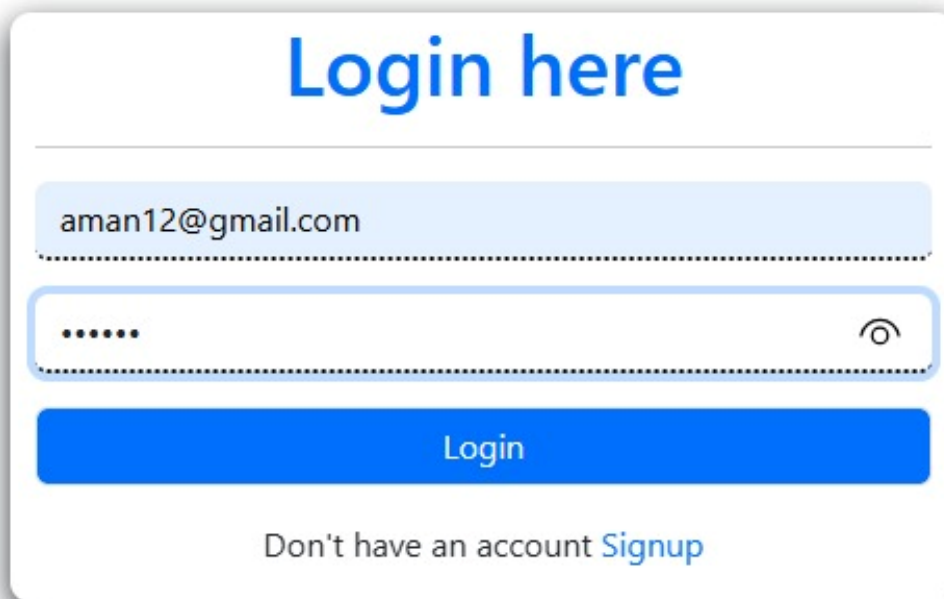
Figure: 6.1.11 Home Page

6.1.2. Registration

Register here

Figure: 6.1.11 Registration

6.1.3. Login



The login form is a white card with a blue border. It features a large blue heading "Login here" at the top. Below it is a light blue input field containing the email "aman12@gmail.com". Underneath is a white password input field with a blue border and a blue eye icon on the right. A solid blue button with the text "Login" is positioned below the password field. At the bottom, the text "Don't have an account" is followed by a blue link "Signup".

Figure: 6.1.3. Login

6.1.4. Dashboard

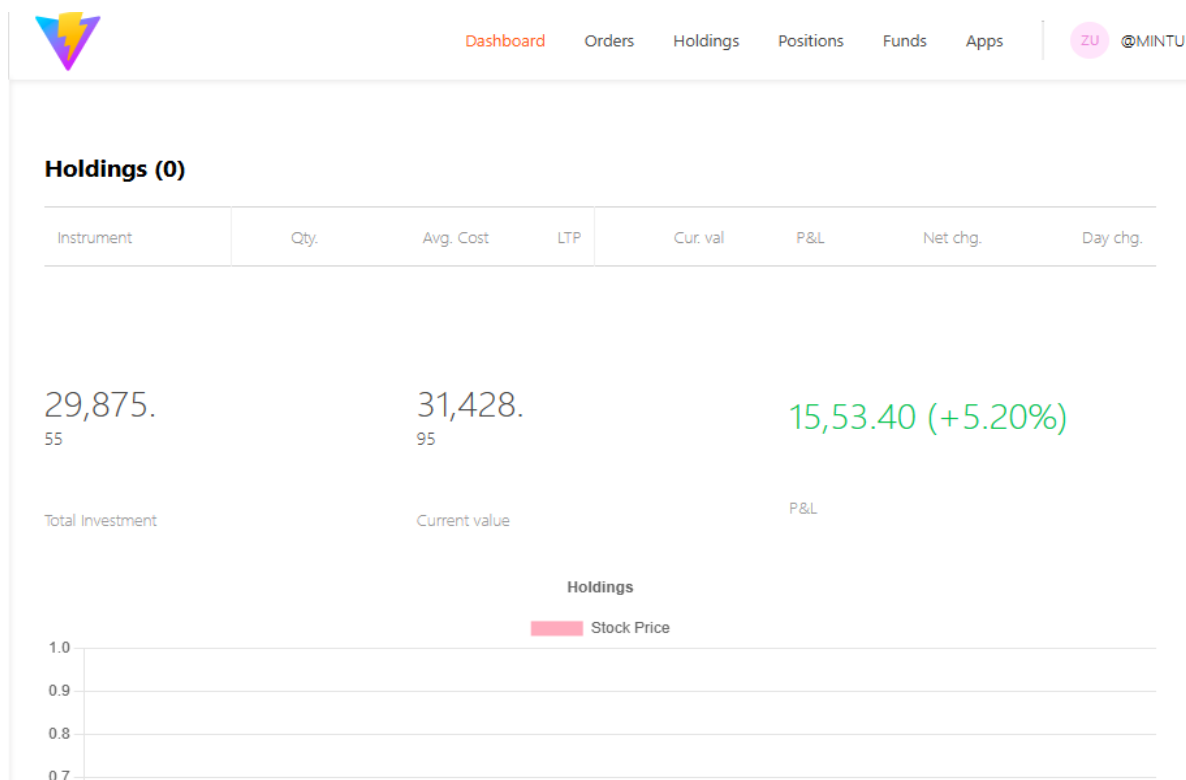


Figure: 6.1.4. Login Dashboard

6.1.5. Stock

NIFTY 50 100.2		SENSEX 100.2	
Search eg:infy, bse, nifty fut weekly, gold mcx			9 / 50
INFY		-1.60%	1555.45
ONGC		-0.09%	116.8
TCS		-0.25%	3194.8
KPITTECH		3.54%	266.45
QUICKHEAL		-0.15%	308.55
WIPRO		0.32%	577.75
M&M		-0.01%	779.8

Figure: 6.1.5. Stock

6.1.6. Stock Chart

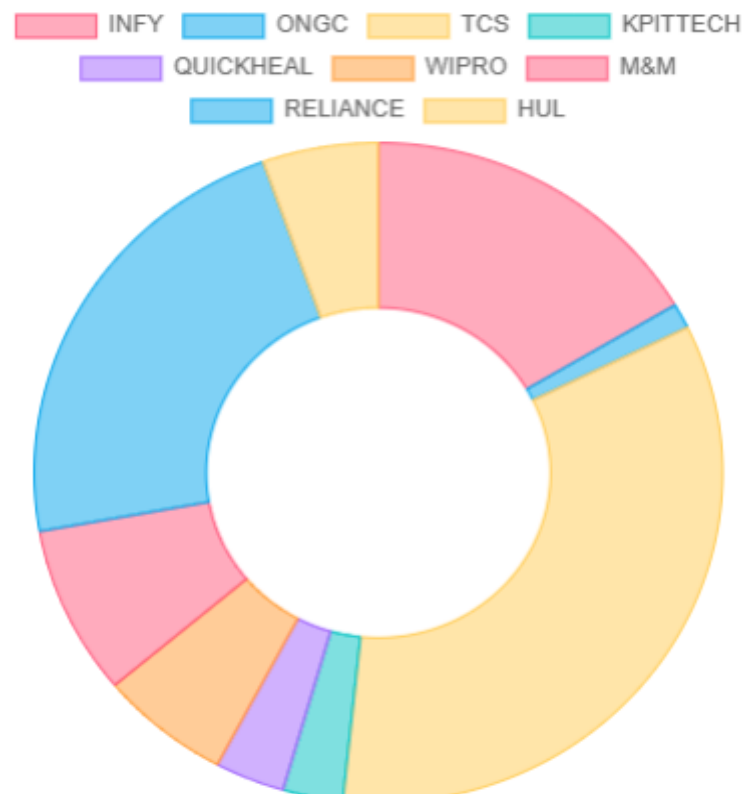


Figure: 7.1.3. Stock Chart

6.2. Discussions

The discussion of the proposed stock trading web application focuses on evaluating the overall performance, usability, and effectiveness of the system after implementation and testing. The developed application successfully demonstrates the core functionalities of a stock trading platform in a simplified and user-friendly manner. The integration of frontend, backend, and database components works efficiently, ensuring smooth communication and data flow across the system. The use of the MERN stack has proven to be effective in building a scalable and responsive application.

From a usability perspective, the system provides an intuitive interface that allows users to easily navigate through different features such as login, stock viewing, trading operations, and portfolio management. The simulated trading environment helps users understand the workflow of stock

trading without involving real financial risks. This makes the system highly beneficial for educational purposes and beginner-level users.

In terms of performance, the system shows satisfactory response time and reliability under normal usage conditions. The modular design of the application simplifies debugging and maintenance, while the use of RESTful APIs ensures efficient communication between different layers. Security measures such as authentication, input validation, and error handling enhance system safety and prevent unauthorized access.

However, the system has certain limitations that can be improved in future versions. The absence of real-time stock market integration limits the accuracy of stock data. Advanced features such as live trading, AI-based recommendations, and detailed analytics are not included in the current implementation. Despite these limitations, the system successfully meets its primary objectives and provides a strong foundation for further development.

Overall, the discussion highlights that the application is effective, reliable, and suitable for learning and demonstration purposes. It reflects the practical implementation of full-stack development concepts and showcases the ability to build a complete web-based trading system using modern technologies.

- Efficient integration of frontend, backend, and database
- User-friendly and easy-to-navigate interface
- Successful implementation of core trading features
- Reliable performance with acceptable response time
- Secure system with authentication and validation
- Useful for learning and simulation purposes
- Modular design improves maintainability

CHAPTER 7: FUTURE SCOPE

The proposed stock trading web application establishes a strong foundation for understanding the working principles of modern trading systems. However, with the continuous evolution of technology and increasing user expectations, there is a wide scope for further enhancements and improvements. Future development of the system can focus on making the platform more advanced, realistic, and suitable for real-world applications by integrating additional features and improving existing functionalities.

One of the most significant areas for future enhancement is the integration of real-time stock market data. Currently, the system operates in a simulated environment, which is useful for learning purposes but lacks real-world accuracy. By integrating external APIs that provide live stock data, the application can offer users up-to-date market information, enabling them to make more realistic and informed trading decisions.

The system can also be improved by incorporating advanced data visualization and analytical tools such as interactive charts, technical indicators, and performance dashboards. Tools like candlestick charts, moving averages, and predictive graphs can significantly enhance decision-making.

Security enhancements such as two-factor authentication (2FA), biometric verification, and end-to-end encryption can further strengthen system protection, especially for real financial systems.

Additional features such as notifications, alerts, and real-time updates can enhance user engagement. The introduction of multi-user roles like admin, trader, and analyst can make the system more robust and organized.

Integration with external financial news APIs and portfolio management tools can provide better insights and improve decision-making.

From a user experience perspective, enhancements like dark mode, improved UI/UX design, and personalization features can make the platform more attractive and user-friendly.

The system can also be extended to support multiple markets, asset types, and global currencies, making it more versatile and suitable for a global audience.

Finally, continuous testing, monitoring, and user feedback collection will ensure ongoing improvement, reliability, and performance optimization.

CHAPTER 8: CONCLUSION

The development of the stock trading web application has successfully demonstrated the practical implementation of a full-stack system using modern web technologies. The project achieved its primary objective of creating a simplified and user-friendly platform that allows users to understand and perform stock trading operations in a simulated environment. By integrating the frontend, backend, and database effectively, the system ensures smooth data flow, efficient processing, and reliable performance.

The application provides essential features such as user authentication, stock viewing, trading operations, and portfolio management, all within an intuitive interface. The use of the MERN stack has enabled the development of a scalable, responsive, and maintainable system. Additionally, the project highlights the importance of modular design, API integration, and secure data handling in building real-world applications.

Although the system has certain limitations, such as the absence of real-time market integration and advanced analytics, it serves as a strong foundation for future enhancements. Overall, this project not only fulfills its intended objectives but also provides valuable insights into the design and development of web-based trading platforms. It reflects the practical application of theoretical knowledge and demonstrates the capability to build efficient and scalable software solutions.

CHAPTER 9: REFERENCES

The references for this project include various research papers, journals, and online resources related to stock trading systems, algorithmic trading, and web application development. Information has been gathered from academic studies in areas such as Machine Learning and Algorithmic Trading, as well as from existing platforms like Zerodha, Upstox, and Groww.

Additional references include official documentation of technologies such as React.js, Node.js, and MongoDB, along with reliable online tutorials and technical articles that helped in understanding system design, development, and implementation of the Zerodha Clone project.

9.1. Books & Study Materials

1. Murphy, J. J. (1999). Technical Analysis of the Financial Markets.
<https://www.pdfdrive.com/technical-analysis-of-the-financial-markets-e17602298.html>
2. [2] Hull, J. C. (2018). Options, Futures, and Other Derivatives.
<https://www.pdfdrive.com/options-futures-and-other-derivatives-e17602253.html>

9.2. Web Technologies Documentation

1. React.js Official Documentation.
Available at: <https://react.dev/learn>
2. Node.js Official Documentation.
Available at: <https://nodejs.org/en/docs>
3. Express.js Documentation.
Available at: <https://expressjs.com/en/guide/routing.html>
4. MongoDB Documentation.
Available at: <https://www.mongodb.com/docs/manual/>
5. REST API Concepts and Guidelines.
Available at: <https://restfulapi.net/>

9.3. Artificial Intelligence & Machine Learning Resources

1. Théate, T. & Ernst, D. (2020).
an Application of Deep Reinforcement Learning to Algorithmic Trading.

Available at: <https://arxiv.org/abs/2004.06627>

2. Liu, X. Y., et al. (2020).
FinRL: A Deep Reinforcement Learning Library for Automated Stock Trading.
Available at: <https://arxiv.org/abs/2011.09607>
3. Pricope, T. V. (2021).
Deep Reinforcement Learning in Quantitative Algorithmic Trading: A Review.
Available at: <https://arxiv.org/abs/2106.00123>
4. Bajpai, S. (2021).
Application of Deep Reinforcement Learning for Indian Stock Trading Automation.
Available at: <https://arxiv.org/abs/2106.16088>
5. Machine Learning Basics – IBM.
Available at: <https://www.ibm.com/topics/machine-learning>